

Import ohne Einfrieren

Organisation: Sie bearbeiten dieses Blatt in Ihrem Zweierteam im Repository aus Blatt 1. Wo eine Aufgabe Rollen nennt, übernimmt eine Person Rolle A und die andere Rolle B. Die erste bis vierte Aufgabe setzen T11A voraus. **Die fünfte bis siebte Aufgabe bearbeiten Sie erst nach T11B, Testat 6 schreiben Sie frühestens nach T11B.** Sie arbeiten mit freiraum-start-6.zip aus Moodle oder mit Ihrem eigenen Stand nach Blatt 5. Aufgaben und Testat 6 verwenden die Klassennamen des Startstands. Alle Räume, Gruppen und Zeiten in den Beispieldaten sind frei erfunden.

Sie schreiben den Java-Code dieses Blatts ohne KI-Werkzeug und bedienen Git selbst, Befehl für Befehl. Im Testat und in der Vorführung erklären Sie jede Zeile und jeden Befehl.

Abgabe: Setzen Sie am Ende den Tag v1.0, packen Sie den Projektordner **mit** dem Ordner .git als ZIP und laden Sie ihn bis [OFFEN: Datum] über Moodle/APA hoch. Messwerte und Fehlermeldungen kopieren Sie wörtlich aus Ihrem eigenen Lauf in NOTIZEN.md, Abschnitt „Blatt 6 · Wartekommunikation und bekannte Mängel“. Die Vorführung bereiten Sie in der siebten Aufgabe vor.

Erste Aufgabe (30 Minuten): Spielen Sie freiraum-start-6.zip wie in Blatt 5 über den Branch start-6 ein und lesen Sie README.md. Neu sind der Konfliktpruefer, die Dateien raeume-semester.csv, semesterplan-gross.csv und semesterplan-kaputt.csv im Ordner daten sowie 200 Räume in der Raumliste. Der Konfliktpruefer bremst jede Prüfung absichtlich, damit der Import so lange dauert wie eine Prüfung über das Netz. Ändern Sie VERZOEGERUNG_MS nicht.

```
public Konfliktpruefer(Campus campus)

/** Liefert null, wenn die Buchung passt, sonst den Grund. Passende Buchungen werden für die folgenden
    Prüfungen vorgemerkt. */
public String pruefe(String raumKennung, Buchung b) {
```

Legen Sie in FreiraumApp einen Button „Semesterplan importieren“ an und schreiben Sie den Import zuerst direkt in dessen Handler: alle Zeilen von daten/semesterplan-gross.csv mit Files.readAllLines lesen, Kommentarzeilen (beginnen mit #) überspringen, jede andere Zeile mit BuchungsDatei.zeileLesen in eine Buchungszeile verwandeln, mit einem Konfliktpruefer für system.getCampus() prüfen und die Konflikte zählen. Am Ende steht die Zahl der Konflikte in der Statuszeile. Gebucht wird noch nichts. Anders als in Blatt 5 rechnet dieser Handler selbst; die Messung im nächsten Absatz braucht genau diesen Stand.

Starten Sie die Oberfläche, klicken Sie auf „Semesterplan importieren“ und stoppen Sie die Sekunden, bis die Konfliktzahl erscheint. Versuchen Sie währenddessen, das Fenster zu verschieben, einen Raum anzuklicken und „Buchen“ zu drücken. Notieren Sie in NOTIZEN.md: die gemessenen Sekunden, was das Fenster zeigt, was das Betriebssystem zeigt (Cursor, Titelzeile, Dialog) und was nach dem Import aus Ihren Klicks wird. Auf dem Referenzrechner kehrte der Handler nach 35,6 Sekunden zurück und fand 109 Konflikte. Commit mit der Nachricht „Import im Handler“.

Zweite Aufgabe (80 Minuten): Der Import zieht aus dem Handler in die Logik und einen Task um. Diese Köpfe sind vorgegeben, Testat 6 verwendet sie:

```
public record Importbericht(int zeilen, List<Buchungszeile> uebernehmbar, List<String> konflikte,
                           List<Fehlerzeile> fehler, long dauerMillis)

public Semesterimport(Campus campus)
```

```
public Semesterimport(Campus campus, int verzoeigerungMs)
public static List<String> zeilenLesen(Path datei) throws IOException
public String verarbeite(int nummer, String zeile)
public Importbericht bericht()

public String uebernehmen(Importbericht bericht) // in Buchungssystem
```

- (A) Branch a-semesterimport: Importbericht und Semesterimport in freiraum.logik. Semesterimport übernimmt die Schleife aus der ersten Aufgabe Zeile für Zeile: verarbeite bekommt die Zeilennummer in der Datei (ab 1) und den Zeilentext, liefert die Konfliktmeldung dieser Zeile oder null und merkt sich übernehmbare Buchungen. Den Campus verändert Semesterimport nicht. Mit verzoeigerungMs 0 läuft die Prüfung ohne künstliche Pause. Ergänzen Sie Buchungssystem um uebernehmen: Die Methode bucht alle übernehmbaren Buchungen des Berichts und liefert eine Meldung für die Statuszeile.
- (B) Branch b-task: Import als Task<Importbericht> in FreiraumApp, gegen die Köpfe oben, bis A fertig ist. call() liest die Datei mit zeilenLesen, ruft für jede Zeile verarbeite auf und meldet nach jeder Zeile updateProgress(...) und updateMessage(...) mit Zeilennummer und Konfliktzahl. call() fasst keinen Knoten an. Im Handler, vor dem Start: eine ProgressBar einmal an progressProperty() binden, die Statuszeile an messageProperty(), „Semesterplan importieren“ und „Buchen“ mit setDisable(true) sperren. setOnSucceeded löst beide Bindungen mit unbind(), entspermt die Buttons und zeigt die Meldung von system.uebernehmen(...). Gestartet wird mit new Thread(task).start().

Mergen Sie beide Branches nach main und prüfen Sie:

- Der Handler kehrt sofort zurück, das Fenster lässt sich während des Imports verschieben, der Balken wächst.
- Am Ende sind 11.721 Buchungen übernommen, 109 Konflikte, 0 fehlerhafte Zeilen. Auf dem Referenzrechner dauerte das 36,3 Sekunden.
- Erscheint java.lang.RuntimeException: Label.text : A bound value cannot be set., setzt Ihr Code die Statuszeile, während sie noch gebunden ist.
- Pendelt der Balken nur hin und her, ohne einen Stand zu zeigen, kommt kein updateProgress an.

Schließen Sie das Fenster, während der Import läuft, und beobachten Sie in IntelliJ oder im Terminal, wann das Programm endet. Auf dem Referenzrechner lebte das Programm nach dem Schließen bei Sekunde 4 insgesamt 36,9 Sekunden. Rufen Sie am Thread vor start() setDaemon(true) auf und wiederholen Sie die Probe. Beide Beobachtungen kommen in NOTIZEN.md. Commit.

Dritte Aufgabe (30 Minuten): Neben der Belegung erscheint eine dritte Liste, `ListView<String>` mit der Überschrift „Konflikte beim Import“. Der Handler leert sie vor dem Start. Jede Konfliktmeldung, die verarbeitet in `call()` liefert, soll sofort in dieser Liste stehen, noch während der Import läuft. `call()` reicht dazu jeden Eintrag mit `Platform.runLater` an den Event-Thread weiter und fasst die Liste selbst nie an. Prüfen Sie: Nach einer Sekunde stehen die ersten Einträge in der Liste, am Ende sind es 109. Auf dem Referenzstand lautet der erste Eintrag Zeile 237: LR-1 ist Mo 08:30–09:30 schon belegt (Team Blau). Schreiben Sie in `NOTIZEN.md` einen Satz dazu, warum `Platform.runLater` nicht um die ganze Schleife gehört. Commit.

Vierte Aufgabe (60 Minuten): Gescheiterter Import und Abbruch.

Datei fehlt: Benennen Sie `daten/semesterplan-gross.csv` vorübergehend um und importieren Sie. Solange `setOnFailed` fehlt, sehen Sie nichts: Der Task endet still im Zustand `FAILED`, und die Exception steckt in `getException()`, auf dem Referenzrechner `java.nio.file.NoSuchFileException: daten/semesterplan-gross.csv`. Registrieren Sie `setOnFailed`: Bindungen lösen, Buttons entsperren, einen Alert vom Typ `ERROR` zeigen. Der Dialogtext beantwortet die drei Fragen aus T06B: Was ist passiert? Was heißt das für mich? Was kann ich jetzt tun? Der Text von `getMessage()` allein reicht dafür nicht. Bewerten Sie Ihren Dialogtext mit der Tafel-Rubrik aus T06B (Moodle) und tragen Sie Text und Punkte in `NOTIZEN.md` ein. Benennen Sie die Datei zurück.

Unlesbare Zeilen: Kopieren Sie `semesterplan-kaputt.csv` vorübergehend über `semesterplan-gross.csv`; `git restore daten/semesterplan-gross.csv` holt das Original zurück. Eine unlesbare Zeile bricht den Import nicht ab. verarbeitet fängt `NumberFormatException` und `IllegalArgumentException`, legt eine Fehlerzeile mit Nummer und Grund in den Bericht, und `setOnSucceeded` hängt die Fehlerzeilen an die Konfliktliste an. Erwartet werden 11.716 übernehmbare Buchungen, 109 Konflikte und 5 fehlerhafte Zeilen:

```
Zeile 347: keine gültige Zahl (For input string: "10")
Zeile 1203: 6 statt 7 Spalten
Zeile 5810: Ende 13:15 liegt nicht nach Beginn 14:15
Zeile 9001: unbekannter Raum LR-999
Zeile 11000: keine gültige Zahl (For input string: "zwölf")
```

Abbruch: Ein Button „Abbrechen“ ist nur während des Imports aktiv und ruft `cancel()` am Task auf. Die Schleife in `call()` fragt in jeder Runde `isCancelled()` ab. Ohne diese Abfrage springt der Zustand auf `CANCELLED`, die Schleife läuft aber weiter. `setOnCancelled` löst die Bindungen und entspermt die Buttons. Entscheiden Sie, ob bei Abbruch nichts übernommen wird oder das bereits Geprüfte, und tragen Sie Entscheidung, Kriterium und Preis in `NOTIZEN.md` ein.

Ohne Fenster: (A) schreibt in `freiraum.logik` das Prüfprogramm `PruefungImport`. Es prüft mit `new Semesterimport(campus, 0)` die kaputte Datei (fünf Fehlerzeilen, die erste mit Nummer 347) und ruft `zeilenLesen` für eine fehlende Datei auf (`NoSuchFileException`). Prüfen Sie dort den `Semesterimport`. Einen Task können Sie aus `main` nicht abfragen, er meldet dann `java.lang.IllegalStateException: Task must only be used from the FX Application Thread`. Commit.

Fünfte Aufgabe (20 Minuten): Erst nach T11B. Brechen Sie die Regel „Die Oberfläche wird nur vom Event-Thread angefasst“ auf dem Branch `regelbruch` gezielt, in zwei Varianten nacheinander:

1. Ersetzen Sie in `call()` die Zeile mit `Platform.runLater` durch den direkten Aufruf `konflikte.getItems().add(konflikt)`.

2. Nehmen Sie Variante 1 zurück. Kommentieren Sie die Bindung der Statuszeile an `messageProperty()` aus und ersetzen Sie in `call()` den Aufruf von `updateMessage(...)` durch `status.setText(...)` mit demselben Text.

Starten Sie jede Variante und lassen Sie den Import bei sichtbarem Fenster laufen. Notieren Sie je Variante wörtlich die erste Zeile, die auf der Konsole erscheint, oder „keine Ausgabe“, dazu, wie der Import endet und ob die Oberfläche richtig aussieht. Erscheint `Label.text : A bound value cannot be set.`, ist die Bindung der Statuszeile noch aktiv. T11 nennt zwei Ausgänge eines Regelbruchs: eine Exception oder stilles Fehlverhalten. Ordnen Sie beide Varianten nach Ihrer Beobachtung zu und schreiben Sie einen Satz dazu, welcher Ausgang für die Menschen vor dem Bildschirm gefährlicher ist. Wechseln Sie zurück nach `main` und löschen Sie den Branch mit `git branch -D regelbruch`.

Sechste Aufgabe (20 Minuten): Erst nach T11B. Ordnen Sie die Rückmeldung Ihres Imports in die Treppe der Wartekommunikation ein: nichts, Cursor, Spinner, Fortschritt mit Restzeit, Skelett-Ansicht, optimistisches UI. Wählen Sie die Stufe, die zu Ihrer gemessenen Importdauer passt, und begründen Sie die Wahl an der Dauer und an den Faustwerten 0,1 s, 1 s und 10 s. Nennen Sie den Preis dieser Stufe für Ihr Team und für die Menschen, die buchen wollen, während der Import läuft. Zeigt Ihre Oberfläche die Stufe noch nicht, ergänzen Sie sie oder notieren Sie das Fehlende als bekannten Mangel. Beobachten Sie bei einer Restzeitanzeige die ersten Sekunden; auf dem Referenzstand stand nach einer Sekunde noch etwa 34 s. Alles kommt in `NOTIZEN.md`. Commit.

Siebte Aufgabe (60 Minuten): Erst nach T11B. Angriff, Abschluss, Vorführung.

- (A) greift den Teil von B an, danach (B) den Teil von A. Jeder Angriff stammt aus einem der vier Cluster aus T13B: Kapselungsbruch, Sonderfall der Datenstruktur, unbehandelte Exception, einfrierender Handler. Schreiben Sie den Angriff als Aufruffolge oder Bedienfolge mit erwartetem und tatsächlichem Ergebnis in `NOTIZEN.md`.
- Wer angegriffen wurde, repariert die Stelle mit eigenem Commit oder trägt sie in `README.md` als bekannten Mangel ein, mit Namen und Grund.

Ergänzen Sie `README.md` um den Start der Oberfläche (`mvn javafx:run` im Projektordner), die Prüfprogramme mit ihrer erwarteten letzten Zeile und die bekannten Mängel. Setzen Sie auf `main` den Tag `v1.0` und pushen Sie ihn mit `git push origin v1.0`.

Bereiten Sie die Vorführung vor. Jede Person beantwortet am eigenen Code zwei Fragen: „Warum ist Buchungssystem eine eigene Klasse?“ und „Was passiert, wenn die Datei fehlt?“. Die zweite Antwort führen Sie live vor, indem Sie `semesterplan-gross.csv` umbenennen und importieren. Dazu kommt eine Frage zu einem Eintrag Ihrer Wahl in `NOTIZEN.md`.